

Administrivia

Concepts of Programming Languages Lecture 0

Course Staff

Instructors: Nathan Mull & Ankush Das (me)

Teaching Fellow: Jared Pincus

Teaching Assistant: Vivian Tvedt

Office Hours are posted on website and course calendar

Course Website

<https://bu-cs320.github.io/cs320-f26/>

Go to the website and read the manual

Grade Breakdown

15%	Assignments	(9 total, 1 dropped)
05%	Final Project	(last 2 weeks)
10%	Labs	(5 total, 1 dropped)
10%	Workshops	(5 total, 1 dropped)
20%	Quizzes in Labs	(5 total, 1 dropped)
25%	Midterm Exams	(October 15 and November 12)
20%	Final Exam	(December 17, Cumulative)

Assignments

- » Assignments include both written and programming components
- » Assignments are due weekly by 8:00PM on Thursdays, one week after the release date. No late submissions (the submission will be open until 11:59PM, with no penalty)
- » Assignments are submitted via **Gradescope**. Regrades are open for a week only
- » Solutions must be your own. No group submissions and no AI assistants
- » Sources must be cited. Any violations will be considered academic misconduct
- » We will automatically drop your lowest assignment score

Lectures

- » We will not take attendance in lectures, but it is highly recommended that you attend
- » If something is said in lecture and not on Piazza there is **no excuse for missing the information**
- » Barring technical issues, lectures will be **recorded**

Workshops

- » Five lectures are labeled as **workshops**, where there will be a **graded** in-class activity
- » We drop your lowest workshop grade (e.g., you're allowed to miss one)
- » *see course schedule for more details*

Labs

- » Labs meet **weekly on Wednesdays**. See the course webpage for meeting times and locations
- » Labs are an opportunity to practice the material
- » **We will take attendance**, insofar as you will (often) have to submit something

Quizzes

- » Five labs are labeled as **quizzes**, where you need to submit a worksheet
- » We drop your lowest quiz grade (e.g., you're allowed to miss one)
- » *see course schedule for more details*

Final Project

- » In the last 2 weeks of the course, you will complete a final project (think of this as a longer assignment)
- » In this project, you will develop an interpreter for a fragment of OCaml
- » You **cannot** drop the final project

Grading

» Programming assignments and final project will be graded with autograders

- **The score you see is the score you get**
- If we can't build your code, **you'll receive a zero**
- Please submit early enough to be able to see the autograder output

» Written assignments are graded by hand

- They must be exceptionally neat
- You must choose the correct pages on Gradescope
- We reserve the right to **deduct points for not following these instructions**

Course Communication

- » We will be using **Piazza** for course communication
- » Please check regularly. "I didn't see the piazza post" is not a valid excuse
- » We will not respond to any material-related questions by email
- » If you have logistical questions (e.g., about disability accommodations) send me an email directly

Course Repository

The course repo contains starter code and lecture material

In **Assignment 1**, you'll set up a mirror of this repository for assignment submission

<https://github.com/BU-CS320/cs320-f26>

Course Standard Library

We'll assume a barebones standard library for the course

You'll need to familiarize yourself with what's there during this first half

You must have installed it in Lab 1

Questions?

If I missed anything, [ask on Piazza](#)

Make sure you're on Piazza and Gradescope,
checking the course webpage, and pulling down
the course repo frequently

**By continuing in this course you're agreeing to all these
conditions**

One Last Thing

Remember to be kind. This is a difficult course. Don't take it out on other students

We care about your success in this course.
We're not out to get you, we're here to help

Use your best judgment, you're adults

Course Introduction

Concepts of Programming Languages
Lecture 1

CAS CS 320

Outline

- » Give an overview of what PL is about
- » Take a first look at OCaml

Theme of the Course

- » This is not a course on “Programming”! We will not teach you programming. In fact, I am not a good programmer myself.
- » This is a course on “Programming Languages” (PL).
- » You’ll learn “**Programming Abstractions**”, i.e., high-level constructs to make programming easier (e.g., loops, functions, arithmetic)

What is a PL?

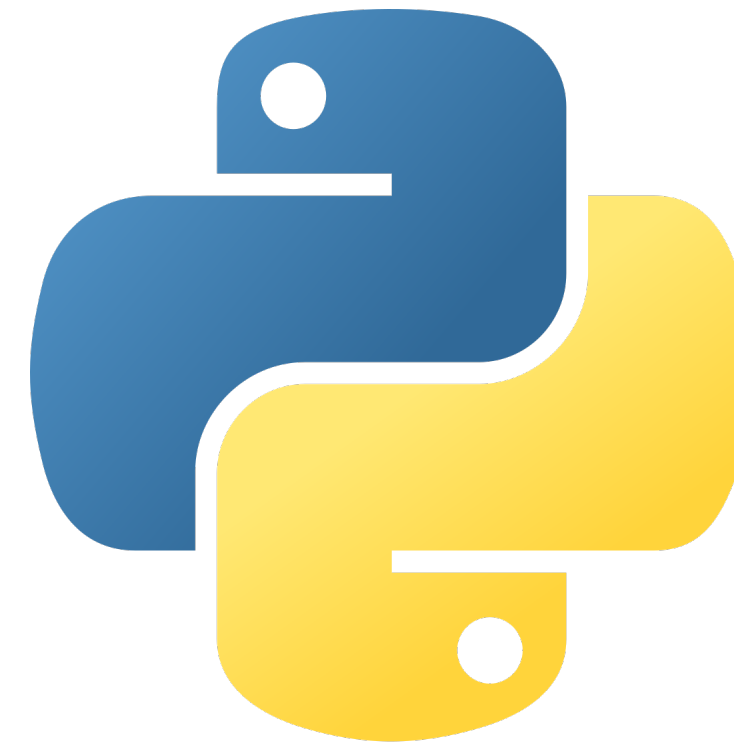
Fair Question

How would you define a PL?

How would you explain it to your roommate?

How would you answer if you were asked during an interview?

Discuss this with the people around you for 1 min



Programmer's view of a PL

- » A tool for programming
- » A text-based way of interacting with hardware/a computer
- » A way of organizing and working with data

This is not what the course is about

```
42 def subtraction():
43     num_1 = randint(0,9)
44     num_2 = randint(0,9)
45
46     print(f"What is {num_1} - {num_2} ?\n")
47
48     choice = input("> ")
49
50     if int(choice) == num_1 - num_2:
51         print("Correct! Nice job! Keep on playing!\n")
52         start_game()
53     else:
54         print(f"Incorrect...the answer is {num_1-num_2}!\n")
55         start_game()
56
57 def multiplication():
58     num_1 = randint(0,9)
59     num_2 = randint(0,9)
60
61     print(f"What is {num_1} x {num_2} ?\n")
62
63     choice = input("> ")
64
65     if int(choice) == num_1*num_2:
66         print("Correct! Nice job! Keep on playing!\n")
67         start_game()
68     else:
69         print(f"Incorrect...the answer is {num_1*num_2}!\n")
70         start_game()
```


Users vs. Designers

Programmers *use* PLs. We're interested in **designing** PLs

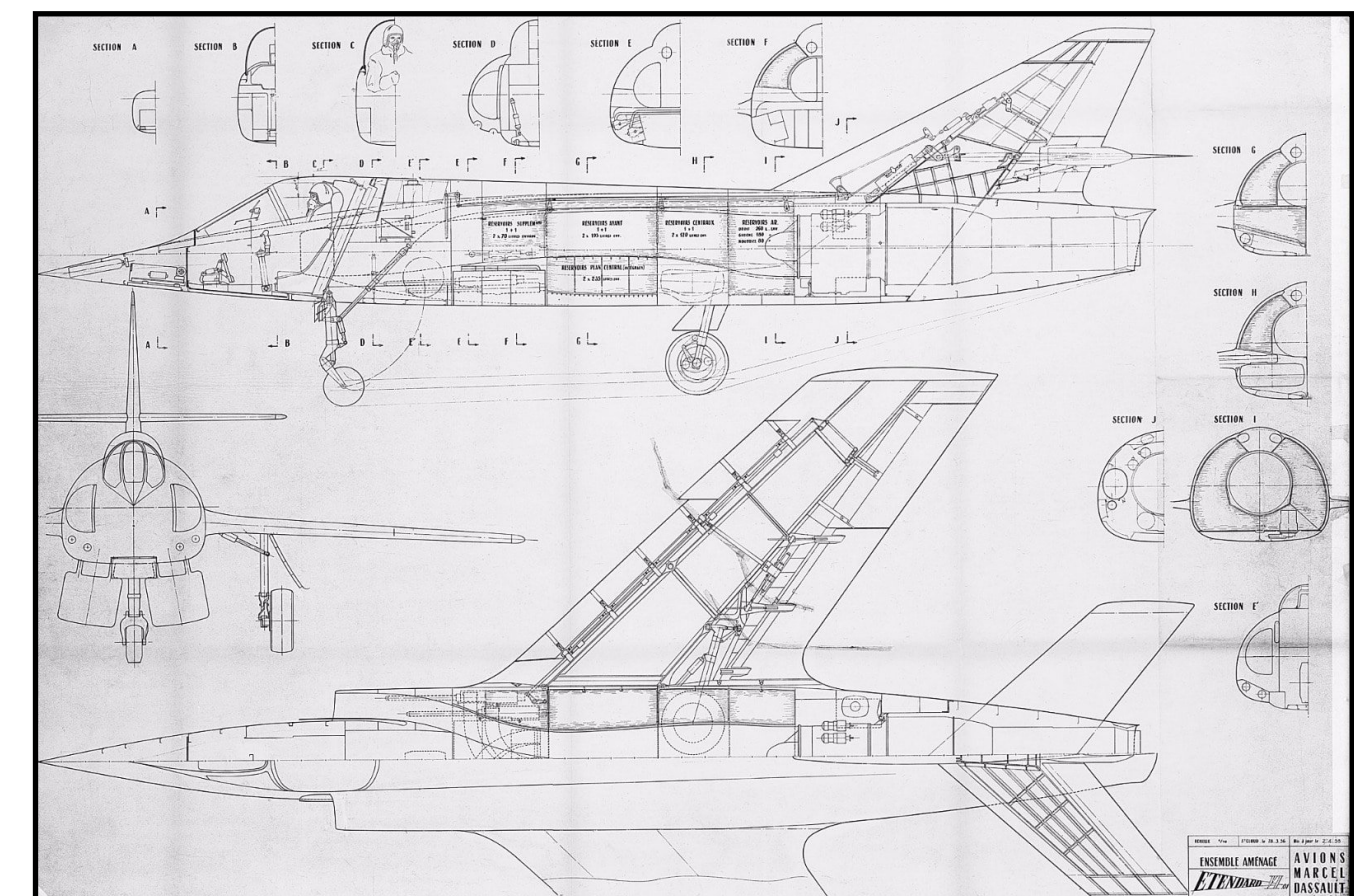
Users are not necessarily the best designers...Who should design PLs?

Answer: Mathematicians

(CS320 is a math class, sorry)



VS.



Mathematician's View of PL

- » a mathematical object, like a polynomial or a vector
- » a formal specification
- » composed of exactly three things:

- Syntax
- Type System
- Semantics

```
42 def subtraction():
43     num_1 = randint(0,9)
44     num_2 = randint(0,9)
45
46     print(f"What is {num_1} - {num_2} ?\n")
47
48     choice = input("> ")
49
50     if int(choice) == num_1 - num_2:
51         print("Correct! Nice job! Keep on playing!\n")
52         start_game()
53     else:
54         print(f"Incorrect...the answer is {num_1-num_2}!\n")
55         start_game()
56
57 def multiplication():
58     num_1 = randint(0,9)
59     num_2 = randint(0,9)
60
61     print(f"What is {num_1} x {num_2} ?\n")
62
63     choice = input("> ")
64
65     if int(choice) == num_1*num_2:
66         print("Correct! Nice job! Keep on playing!\n")
67         start_game()
68     else:
69         print(f"Incorrect...the answer is {num_1*num_2}!\n")
70         start_game()
```

VS.

Syntax		Evaluation	
$t ::=$			$t \rightarrow t'$
x	terms:		
$\lambda x:T. t$	variable	$\frac{t_1 \rightarrow t'_1}{t_1 t_2 \rightarrow t'_1 t_2}$	(E-APP1)
$t t$	abstraction	$\frac{t_2 \rightarrow t'_2}{v_1 t_2 \rightarrow v_1 t'_2}$	(E-APP2)
$v ::=$	values:	$(\lambda x:T_{11}. t_{12}) v_2 \rightarrow [x \mapsto v_2] t_{12}$	(E-APPABS)
$\lambda x:T. t$	abstraction value	Typing	
			$\Gamma \vdash t : T$
$T ::=$	types:	$\frac{x:T \in \Gamma}{\Gamma \vdash x : T}$	(T-VAR)
$T \rightarrow T$	type of functions	$\frac{\Gamma, x:T_1 \vdash t_2 : T_2}{\Gamma \vdash \lambda x:T_1. t_2 : T_1 \rightarrow T_2}$	(T-ABS)
$\Gamma ::=$	contexts:	$\frac{\Gamma \vdash t_1 : T_{11} \rightarrow T_{12} \quad \Gamma \vdash t_2 : T_{11}}{\Gamma \vdash t_1 t_2 : T_{12}}$	(T-APP)
$\emptyset$	empty context		
$\Gamma, x:T$	term variable binding		

(from TAPL by Pierce)

Why does this matter?

There are a lot of *bad* PLs out there

We want *good* PLs

This course is about **what makes a PL good**

The punchline: mathematically well-defined
syntax, type system, and semantics

Formal PL

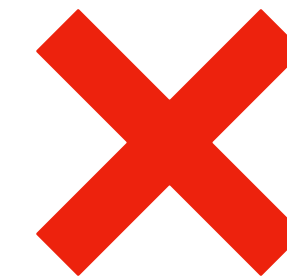
The Three Components

Syntax: What a *well-formed* program in your PL?

```
def f():  
    return 3
```



```
define f():  
    3 return
```

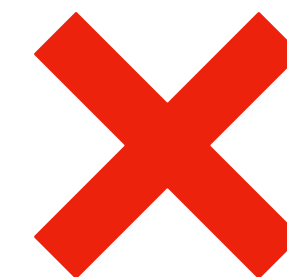


Type System (Static Semantics): What is a *valid* program in your PL?

```
x = 2 + 2
```



```
x = 2 + "two"
```

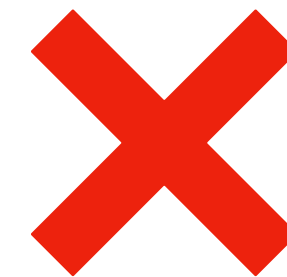


Semantics (Dynamic Semantics): What is the *output* of a (valid) program?

```
>>> 2 + 2  
4
```



```
>>> 2 + 2  
False
```



For everything we do from now on,
we'll define the syntax rules, the
typing rules, and the semantic rules

Syntax Rules

Syntax rules describe well-formed expressions.

They are independent of meaning

A syntax rule of the form:

*If **<such-and-such>** is a well-formed expression and **<some-other-things>** are a well-formed expression, then **<some-combination-of-such-and-such-and-some-other-things>** is a well-formed expression*

Example: Integer Addition Syntax

If e_1 is a well-formed expression and e_2 is a well-formed expression, then $e_1 + e_2$ is a well-formed expression

Formal notation:

$\langle \text{expr} \rangle ::= \langle \text{expr} \rangle + \langle \text{expr} \rangle$

Typing Rules

Typing rules describe the types of expressions

They are of the form:

If $\langle \text{such-and-such} \rangle$ is of $\langle \text{such-and-such-type} \rangle$ and $\langle \text{some-other-things} \rangle$ are of $\langle \text{some-other-types} \rangle$, then $\langle \text{some-combination-of-such-and-such-and-some-other-things} \rangle$ is of $\langle \text{some-different-type} \rangle$

Example: Integer Addition Typing

*If e_1 is an **int** (in any context Γ) and e_2 is an **int** then (in any context Γ) $e_1 + e_2$ is an **int** (in any context Γ)*

Formal notation:

$$\frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 + e_2 : \text{int}} \text{ (addInt)}$$

Semantic Rules

Semantic rules describe the *value* of an expression

They are of the form:

If **<such-and-such>** evaluates to **<such-and-such-value>**
and **<some-other-things>** evaluate to **<some-other-values>**
then **<some-combination-of-such-and-such-and-some-other-things>**
evaluates to **<some-other-value-computed-based-on-such-and-such-value-and-some-other-values>**

Example: Integer Addition Semantics

If e_1 evaluates to v_1 and e_2 evaluates to v_2 , then $e_1 + e_2$ evaluates to $v_1 + v_2$

Formal Notation:

$$\frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2}{e_1 + e_2 \Downarrow v_1 + v_2} \text{ (evalInt)}$$

We '11 come back to all
this soon...

What Language to Use for the Course?



What is OCaml?



- » It is an "industrial-strength **functional** programming language" with powerful type-inference
- » It was developed at Inria and used/developed heavily by Jane Street (and too a lesser degree by Meta, Microsoft, Docker, Wolfram)
- » It won the ACM SIGPLAN Programming Languages Software Award in 2023

What does Functional Mean?

» A completely different paradigm. We're **not writing procedures via commands/statements**, we're **defining values via expressions**

Functional vs. Imperative

Functional here means:

- » No state (which means no loops!)
- » We don't think of a program as describing a procedure, but as defining a value

State

```
def fact(n):  
    acc = 1  
    for i in range(1, n + 1):  
        acc *= i # acc is "stateful"  
    return acc
```

In Python, we can define variables that change throughout the evaluation of the program

We can't do this in OCaml. Instead we use **recursion**(!)

Imperative Programming

```
def fact(n):  
    acc = 1  
    for i in range(1, n + 1):  
        acc *= i  
    return acc
```

Recursive Programming

```
def fact(n):  
    if n == 0:  
        return 1  
    else:  
        return n * fact(n-1)
```

If you can write recursive
functions in Python, then you
will be able to program in OCaml

Working with OCaml

Anatomy of an OCaml Program

```
let x = 3

let y = "string"

(* function definition *)
let square x = x * x

(* recursive function definition *)
let rec f x = if x = 0 then 0 else x + f (x - 1)

(* We can't just print , we assign to wildcard *)
let _ = print_endline("Hello world")
```

An OCaml Program consists of *top-level let-expressions*
(these are **not** "global variables")

Expressions

Expressions are syntactic objects which describe values in a program

Mnemonic: *Expressions are EValuated to Values*

They appear in both functional and imperative PLs, but in functional PLs we *only* have expressions

$$2 + (2 * 3)$$

if x = 3 then 3 else 4

$$H(f(f(f(x, y), 2), g(z)))$$

Values

Values are the *things* manipulated and output by programs, e.g., the integer 7 or the string "seven"

Expressions *describe* values (the values to which they evaluate)

Example: The expression $2 + 7$ "describes" the value 9

Types

```
let x : int = 2
let y : string = "two"
let _ = x + y (* THIS IS NOT POSSIBLE *)
```

3 | let _ = x + y (* THIS IS NOT POSSIBLE *)
 ^

Error: This expression has type string but an expression was expected of type
int

Every expression in OCaml has a type

The type of an expression describes what *kind* of thing it is

Types **restrict** how expressions can be constructed

Basic Expressions

» Literals

» Let-expressions (local variables)

» If-expressions

» Functions

» Applications

Basic Expressions

» Literals

» Let-expressions (local variables)

» If-expressions

» Functions

» Applications

Primitive Types and Literals

OCaml has a collection of standard literals and types

Type	Literals	Operators
int	0, -2, 13, -023	+, -, *, /, mod
float	3., -1.01	+. , -. , *. , /.
bool	true, false	&&, , not
char	'b', 'c'	
string	"word", "@*&#"	^

A Couple Note on Operators

Operators `int` and `float` are *different*, e.g., `+` (integer addition) and `+.` (float addition)

OCaml has **no operator overloading**

Comparison operators are standard, e.g., `<`, `<=`, `>`, `>=`, and can be used to compare any expressions of the same type

Note that equality check is just `=` (not `==`) and inequality is `<>` (not `!=`)

Basic Expressions

» Literals

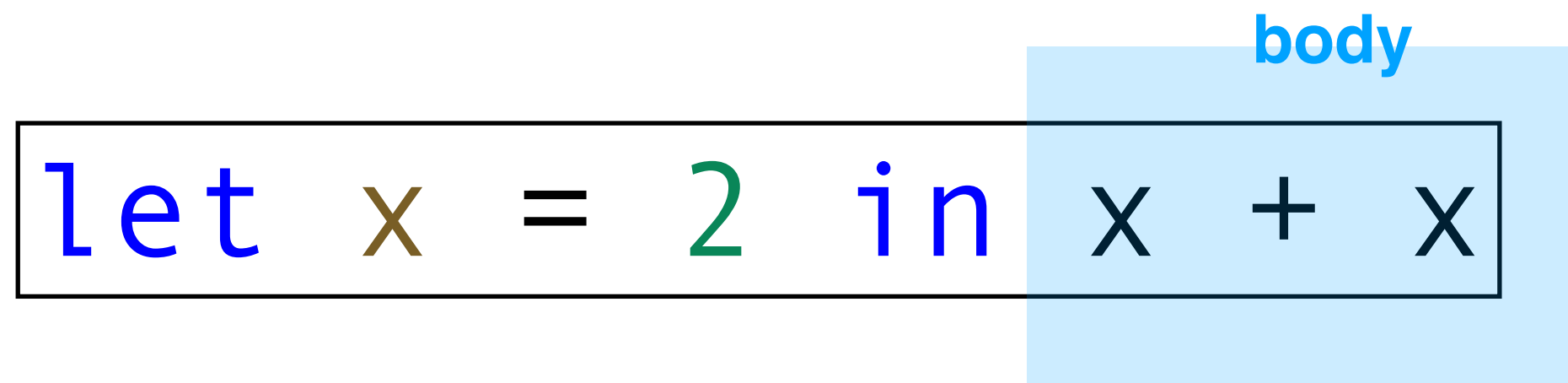
» **Let-expressions (local variables)**

» If-expressions

» Functions

» Applications

Local Variables



The diagram shows the OCaml expression `let x = 2 in x + x` enclosed in a black rectangular box. A light blue rectangular box highlights the right portion of the expression, specifically `x + x`. The word "body" is written in blue text above the light blue box, indicating that the expression to the right of the `in` keyword is the body of the local variable definition.

We can define local variables in OCaml

This is useful for writing better abstractions

Note that it reads like a sentence: *let x stand for 2 in the expression $x + x$*

Multiple Local Variables

```
def sum_of_squares(x, y):  
    x_squared = x * x  
    y_squared = y * y  
    return x_squared + y_squared
```

Python

```
let sum_of_squares x y =  
    let x_squared = x * x in  
    let y_squared = y * y in  
    x_squared + y_squared
```

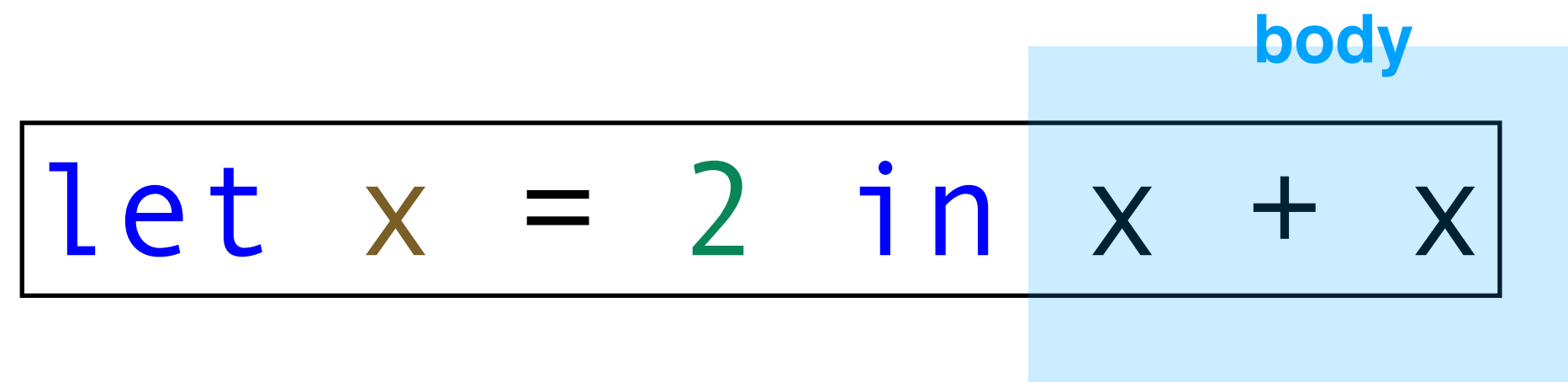
OCaml

It's very easy to use multiple local variables, we just *nest* local variables

(If it helps, think of **in** as a semicolon ;)

IMPORTANT: `let x = e1 in e2` is an *expression* so it can be the body of a let expression

Local Variables (Informal)



The diagram shows the code `let x = 2 in x + x` enclosed in a thin black rectangular border. The word `let` is blue, `x` is brown, `=` is black, `2` is green, `in` is blue, and `x + x` is black. A light blue rectangular box highlights the entire expression `x + x`. Above this box, the word `body` is written in blue.

syntax: `let VARIABLE = EXPRESSION in BODY`

typing: the type is the same as that of `BODY` *given `BODY` is well-typed after substituting the `VARIABLE` in `BODY`*

semantics: the is the same as the value of `BODY` *after substituting the `VARIABLE` in `BODY`*

A Note on Substitution

$$\boxed{\text{let } x = 2 \text{ in } x + x} \longrightarrow \boxed{2 + 2}$$

Formally, we write $[v/x]e$ to mean "substitute v for x in e ", e.g., $[3/x](x + x)$ is the same as $3 + 3$

Intuitively, substitution is simple: **replace the variable**

Turns out, this is **very hard** to do correctly, *it's subtle* and a source of a lot of mistakes in PL implementations

Basic Expressions

» Literals

» Let-expressions (local variables)

» **If-expressions**

» Functions

» Applications

If-Expressions

```
let abs x = if x > 0 then x else -x
```

Note: OCaml is whitespace agnostic!

OCaml has expressions for conditional reasoning

Note: The **else** case is *required* and the **then** and **else** cases must be the *same type* (why?)

If-Expressions

```
let foo x =  
  if x < 0 then  
    "negative"  
  else if x = 0 then  
    "zero"  
  else  
    "positive"
```

Answer: Remember, all we have is expressions. So every if-expression must have a value and a type (and therefore, an **else** case of the same type)

We can do **else if** just by nesting if-expressions! (neat)

If-Expressions (Informal)

```
let abs x = if x > 0 then x else -x
```

Syntax: if CONDITION then TRUE-CASE else FALSE-CASE

Typing: CONDITION must be a Boolean and TRUE-CASE and FALSE-CASE must be the same type. The type is then the same as that of TRUE-CASE and FALSE-CASE

Semantics: If CONDITION holds, then we get the TRUE-CASE, otherwise we get the FALSE-CASE

Basic Expressions

» Literals

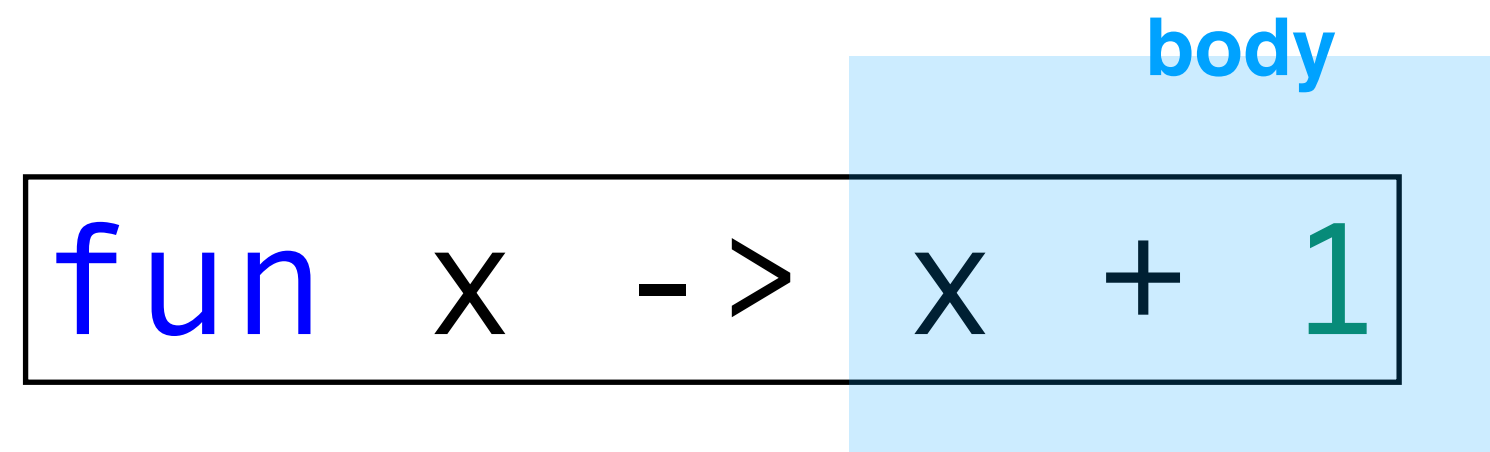
» Let-expressions (local variables)

» If-expressions

» **Functions**

» Applications

Functions (Informal)



```
fun x -> x + 1
```

Syntax: `fun VAR-NAME -> EXPR`

Typing: the type of a function is **`T1 -> T2`** where **`T1`** is the type of the input and **`T2`** is the type of the output

Semantics: A function will evaluate to special *function value* (printed as `<fun>` by utop)

Important: Curried Functions

```
let f = fun x -> fun y -> fun z -> x + y + z
```

The only kind of function we have is *single argument*

This seems restrictive, but ultimately it doesn't affect us at all

We can *simulate* multi-argument functions with nested functions. This is called **Currying** after Haskell Curry

Important: Curried Functions

```
let f = fun x -> fun y -> fun z -> x + y + z
```

We should think of the above function as something which takes an input and returns **another function**

In other words, we *partially apply* the function

Basic Expressions

» Literals

» Let-expressions (local variables)

» If-expressions

» Functions

» **Applications**

Application

```
(fun x -> fun y -> x + y + 1) 3 2
```

Application is done by *juxtaposition* which means we put the arguments next to the function

Application is *left-associative*, which means we pass arguments from left to right

Application (Informally)

```
(fun x -> fun y -> x + y + 1) 3 2
```

Syntax: FUNCTION-EXPR ARG-EXPR

Typing: If FUNCTION-EXPR is of type $T1 \rightarrow T2$,
and ARG-EXPR is of type $T1$, then the type is $T2$

Semantics: Substitute the value of ARG-EXPR into
the body of FUNCTION-EXPR and evaluate that

Application (Example)

(fun x -> fun y -> x + y + 1) 3 2

is the same as

(fun x -> (fun y -> x + y + 1)) 3 2

is the same as

((fun x -> (fun y -> x + y + 1)) 3) 2

evaluates to

(fun y -> 3 + y + 1) 2

evaluates to

3 + 2 + 1

is the same as

(3 + 2) + 1

evaluates to

5 + 1

evaluates to

6

One Last Point: Building Interpreters

PL is math but we still like to *use* PLs. The three components of a PL correspond to the three things we need to *implement* in an **interpreter** of a PL.

» **Syntax** is implemented by a **parser**

```
parse : string -> expr
```

» **Type system** is implemented by a **type checker**

```
type_check : expr -> bool (* valid or not *)
```

» **(Dynamic) semantics** is implemented by an **evaluator**

```
eval : expr -> value
```


Next Steps

- » Make sure you're on Piazza and Gradescope, keep an eye on announcements
- » Bookmark the course webpage and course repo
- » Install opam, VSCode, the course standard library, etc. (*see assignment 1*)
- » **Do the reading listed on the course webpage**

Summary

A PL is a mathematical object given by its **syntax, type system and semantics**

There is **no state** in functional programming.
Programs define the output for a given input

Practice, practice, practice. Functional programming takes time to learn, but once you get it, it's as easy as programming in any other PL